SCALABILITY & FUTURE PLAN

Team ID: unm293200909 **Project Name:** FitBuddy – AI Fitness Plan Generator using Gemini Models

Team Member	Assigned Work
Kavyasri.A	Model selection & architecture; core functionality development; FastAPI routing and user-input processing; routes.py application logic; frontend/Jinja2 development; local deployment and testing; scalability and future planning.

Project Basis

FitBuddy is a web-based application that uses Google Gemini models to generate personalized 7-day workout plans and nutrition or recovery tips. It uses FastAPI for backend routing, Jinja2/HTML for the interface, and SQLite with SQLAlchemy for persistent storage. Users can also submit feedback so the AI can revise an existing workout plan.

Current System Limitations

S.No	Limitation	Impact	Priority
1	The current application uses SQLite for persistence.	May become less suitable as concurrent users and stored plans increase.	Medium
2	The application is designed for local FastAPI/Uvicorn deployment.	Scaling to a large number of simultaneous users would require production infrastructure.	High
3	AI generation depends on Gemini API calls.	Higher traffic can increase API usage, latency, and dependency on external AI service availability.	High

Scalability Plan

S.No	Scalability Aspect	Current State	Proposed Upgrade / Solution
1	User Load	FastAPI/Uvicorn local deployment.	Deploy behind a production ASGI setup/load balancer and scale application instances as demand grows.
2	Data Storage	SQLite with SQLAlchemy ORM stores user details and original/updated plans.	Move to a production relational database such as PostgreSQL when data volume and concurrent access increase.
3	Performance	Gemini Pro is used for workout generation/update and Gemini Flash for fast nutrition tips.	Use caching where appropriate, asynchronous processing, request controls, and efficient prompts to reduce latency and API load.
4	Security	Gemini API key is configured through environment variables.	Use secure secret management, HTTPS, authentication/authorization for admin functions, input validation, logging, and access controls.

Future Roadmap

Planned enhancements for FitBuddy beyond the current local application.

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 1	Production-ready deployment, stronger validation, secure configuration, and improved error handling.	Short term	More reliable and secure application operation.
Phase 2	Database migration from SQLite to a scalable relational database and improved data management.	Medium term	Better support for larger user bases and concurrent access.
Phase 3	Add richer personalization such as progress tracking, plan history, and more adaptive feedback-based recommendations.	Medium term	Improves engagement and long-term personalization.
Phase 4	Cloud deployment and scalable infrastructure for broader access, monitoring, and future integrations.	Long term	Supports growth, availability, and expansion of the FitBuddy platform.

Assigned Work – Kavyasri.A

Order	Responsibility
1	Research and select the appropriate Gemini model and define the FitBuddy architecture.
2	Set up the Python/FastAPI development environment and required libraries.
3	Develop workout-plan generation, nutrition/recovery tip generation, feedback-based plan updating, and user/plan storage.
4	Implement FastAPI routes and main application logic in routes.py.
5	Design the HTML/CSS interface and create dynamic Jinja2 templates.
6	Prepare and verify local deployment using Uvicorn, including API documentation/testing.
7	Document scalability limitations, proposed upgrades, and future roadmap.

Note: The project details and current-state information in this document are based on the uploaded FitBuddy project document. The scalability upgrades and roadmap are proposed planning items for future development.